

PROJET 4 DATA ANALYST

Réalisez une étude de santé publique avec R ou Python

OBJECTIF DE CE NOTEBOOK

Bienvenue dans l'outil plébiscité par les analystes de données Jupyter.

Il s'agit d'un outil permettant de mixer et d'alterner codes, textes et graphique.

Cet outil est formidable pour plusieurs raisons:

- il permet de tester des lignes de codes au fur et à mesure de votre rédaction, de constater immédiatement le résultat d'une instruction, de la corriger si nécessaire.
- De rédiger du texte pour expliquer l'approche suivie ou les résultats d'une analyse et de le mettre en forme grâce à du code html ou plus simple avec **Markdown**
- d'agrémenter de graphiques

Pour vous aider dans vos premiers pas à l'usage de Jupyter et de Python, nous avons rédigé ce notebook en vous indiquant les instructions à suivre.

Il vous suffit pour cela de saisir le code Python répondant à l'instruction donnée.

Vous verrez de temps à autre le code Python répondant à une instruction donnée mais cela est fait pour vous aider à comprendre la nature du travail qui vous est demandée.

Et garder à l'esprit, qu'il n'y a pas de solution unique pour résoudre un problème et qu'il y a autant de résolutions de problèmes que de développeurs ;)...

Etape 1 - Importation des librairies et chargement des fichiers

1.1 - Importation des librairies

```
#Importation de la librairie Pandas  
import pandas as pd
```

1.2 - Chargement des fichiers Excel

```
#Importation du fichier population.csv
population = pd.read_csv('population.csv')

#Importation du fichier dispo_alimentaire.csv
dispo_alimentaire = pd.read_csv('dispo_alimentaire.csv', sep=";")

#Importation du fichier aide_alimentaire.csv
aide_alimentaire = pd.read_csv('aide_alimentaire.csv')

#Importation du fichier sous_nutrition.csv
sous_nutrition = pd.read_csv('sous_nutrition.csv')

# Uniformisation de la dénomination de 5 pays

corrections_geo = {
    "Swaziland" : "Eswatini",
    "Chine- continentale" : "Chine, continentale",
    "Chine- Taiwan Province de" : "Chine, Taiwan Province de",
    "Royaume-Uni" : "Royaume-Uni de Grande-Bretagne et d'Irlande du Nord",
    "Tchéquie (la)" : "Tchéquie"
}

# Implantation des modifications dans les trois fichiers originaux
dispo_alimentaire['Zone'] = dispo_alimentaire['Zone'].replace(corrections_geo)
aide_alimentaire['Pays bénéficiaire'] = aide_alimentaire['Pays
bénéficiaire'].replace(corrections_geo)
sous_nutrition['Zone'] = sous_nutrition['Zone'].replace(corrections_geo)
```

Etape 2 - Analyse exploratoire des fichiers

2.1 - Analyse exploratoire du fichier population

```
#Afficher les dimensions du dataset
print("Le tableau comporte {} observation(s) ou
article(s)".format(population.shape[0]))
print("Le tableau comporte {} colonne(s)".format(population.shape[1]))
```

```
Le tableau comporte 1416 observation(s) ou article(s)
Le tableau comporte 3 colonne(s)
```

```
#Consulter le nombre de colonnes
#La nature des données dans chacune des colonnes
#Le nombre de valeurs présentes dans chacune des colonnes
print("Le nombre de colonnes est de {}".format(len(population.columns)))
```

```
print("Les types de données dans chaque colonne sont
:\n{}".format(population.dtypes))
print("Pour chaque colonne, voici le nombre de valeurs
:\n{}".format(population.count()))
```

```
Le nombre de colonnes est de 3
Les types de données dans chaque colonne sont :
Zone      object
Année      int64
Valeur     float64
dtype: object
Pour chaque colonne, voici le nombre de valeurs :
Zone      1416
Année      1416
Valeur     1416
dtype: int64
```

```
#Affichage les 5 premières lignes de la table
population.head()
```

```
border="1" class="dataframe">
```

	Zone	Année	Valeur
0	Afghanistan	2013	32269.589
1	Afghanistan	2014	33370.794
2	Afghanistan	2015	34413.603
3	Afghanistan	2016	35383.032
4	Afghanistan	2017	36296.113

```
#Nous allons harmoniser les unités. Pour cela, nous avons décidé de multiplier la
population par 1000
#Multiplication de la colonne valeur par 1000
population['Valeur']=population['Valeur']*1000
population.head()
```

```
border="1" class="dataframe">
```

	Zone	Année	Valeur
0	Afghanistan	2013	32269589.0
1	Afghanistan	2014	33370794.0
2	Afghanistan	2015	34413603.0
3	Afghanistan	2016	35383032.0
4	Afghanistan	2017	36296113.0

```
#changement du nom de la colonne Valeur par Population
population=population.rename(columns={'Valeur': 'Population'})
population['Population'] = population['Population'].astype(int)
```

```
#Affichage les 5 premières lignes de la table pour voir les modifications
population.head()
```

border="1" class="dataframe">

	Zone	Année	Population
0	Afghanistan	2013	32269589
1	Afghanistan	2014	33370794
2	Afghanistan	2015	34413603
3	Afghanistan	2016	35383032
4	Afghanistan	2017	36296113

2.2 - Analyse exploratoire du fichier disponibilité alimentaire

```
#Afficher les dimensions du dataset
print("Le tableau comporte {} observation(s) ou
article(s)".format(dispo_alimentaire.shape[0]))
print("Le tableau comporte {} colonne(s)".format(dispo_alimentaire.shape[1]))
```

```
Le tableau comporte 15605 observation(s) ou article(s)
Le tableau comporte 18 colonne(s)
```

```
#Consulter le nombre de colonnes
print("Le nombre de colonnes est de {}".format(len(dispo_alimentaire.columns)))

print("\nLes types de données dans chaque colonne sont
:\n{}".format(dispo_alimentaire.dtypes))

print("\nPour chaque colonne, voici le nombre de valeurs
:\n{}".format(dispo_alimentaire.count()))
```

```
Le nombre de colonnes est de 18
```

```
Les types de données dans chaque colonne sont :
```

Zone	object
Produit	object
Origine	object
Disponibilité alimentaire (Kcal/personne/jour)	float64
Disponibilité alimentaire en quantité (kg/personne/an)	float64
Disponibilité de matière grasse en quantité (g/personne/jour)	float64
Disponibilité de protéines en quantité (g/personne/jour)	float64
Disponibilité intérieure	float64
Exportations - Quantité	float64
Importations - Quantité	float64
Nourriture	float64
Pertes	float64
Production	float64
Semences	float64
Traitement	float64
Aliments pour animaux	float64
Autres Utilisations	float64

Variation de stock	float64
dtype: object	
Pour chaque colonne, voici le nombre de valeurs :	
Zone	15605
Produit	15605
Origine	15605
Disponibilité alimentaire (Kcal/personne/jour)	14241
Disponibilité alimentaire en quantité (kg/personne/an)	14015
Disponibilité de matière grasse en quantité (g/personne/jour)	11794
Disponibilité de protéines en quantité (g/personne/jour)	11561
Disponibilité intérieure	15382
Exportations - Quantité	12226
Importations - Quantité	14852
Nourriture	14015
Pertes	4278
Production	9180
Semences	2091
Traitement	2292
Aliments pour animaux	2720
Autres Utilisations	5496
Variation de stock	6776
dtype: int64	

```
#Affichage les 5 premières lignes de la table
dispo_alimentaire.head()
```

border="1" class="dataframe">

	Zone	Produit	Origine	Disponibilité alimentaire (Kcal/personne/jour)	Disponibilité alimentaire en quantité (kg/personne/an)	Disponibilité de matière grasse en quantité (g/personne/jour)	Disponibilité de protéines en quantité (g/personne/jour)	Disponibilité intérieure	Exportations - Quantité	Importations - Quantité	No urriture	Pertes	Production	Semences	Traitement	Aliments pour animaux	Autres Utilisations	Variation de stock
0	Afg hanistan	Abats Comestible	animale	5.0	1.72	0.20	0.77	53.0	NaN	NaN	53.0	NaN	53.0	NaN	NaN	NaN	NaN	NaN
1	Afg hanistan	Agumes-Autres	vegetale	1.0	1.29	0.01	0.02	41.0	2.0	40.0	39.0	2.0	3.0	NaN	NaN	NaN	NaN	NaN
2	Afg hanistan	Aliments pour enfants	vegetale	1.0	0.06	0.01	0.03	2.0	NaN	2.0	2.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN
3	Afg hanistan	Ananas	vegetale	0.0	0.00	NaN	NaN	0.0	NaN	0.0	0.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN
4	Afg hanistan	Bananes	vegetale	4.0	2.70	0.02	0.05	82.0	NaN	82.0	82.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN

```
#remplacement des NaN dans le dataset par des 0
dispo_alimentaire = dispo_alimentaire.fillna(0)
```

```
#multiplication de toutes les lignes contenant des milliers de tonnes en Kg
colonnes_a_multiplier = ['Disponibilité intérieure', 'Exportations - Quantité',
'Importations - Quantité',
                        'Nourriture', 'Pertes', 'Production', 'Semences',
'Traitement',
                        'Aliments pour animaux', 'Autres Utilisations', 'Variation
de stock']
dispo_alimentaire[colonnes_a_multiplier] =
dispo_alimentaire[colonnes_a_multiplier].astype('int64')
dispo_alimentaire[colonnes_a_multiplier] = dispo_alimentaire[colonnes_a_multiplier]
*1000000
```

```
#Affichage les 5 premières lignes de la table
```

```
dispo_alimentaire.head()
```

border="1" class="dataframe">

	Zone	Produit	Origine	Disponibilité alimentaire (Kcal/personne/jour)	Disponibilité alimentaire en quantité (kg/personne/jour)	Disponibilité de matière grasse en quantité (g/personne/jour)	Disponibilité de protéines en quantité (g/personne/jour)	Disponibilité intérieure	Exportations - Quantité	Importations - Quantité	Noirure	Pertes	Production	Semences	Traitement	Aliments pour animaux	Autres Utilisations	Variation de stock
0	Afghanistan	Abats Comestible	animale	5.0	1.72	0.20	0.77	530000	0	0	530000	0	530000	0	0	0	0	0
1	Afghanistan	Agumes-Autres	végétale	1.0	1.29	0.01	0.02	410000	200000	400000	390000	200000	300000	0	0	0	0	0
2	Afghanistan	Aliments pour enfants	végétale	1.0	0.06	0.01	0.03	200000	0	200000	200000	0	0	0	0	0	0	0
3	Afghanistan	Ananas	végétale	0.0	0.00	0.00	0.00	0	0	0	0	0	0	0	0	0	0	0
4	Afghanistan	Bananes	végétale	4.0	2.70	0.02	0.05	820000	0	820000	820000	0	0	0	0	0	0	0

2.3 - Analyse exploratoire du fichier aide alimentaire


```
#Afficher les dimensions du dataset
print("Le tableau comporte {} observation(s) ou
article(s)".format(aide_alimentaire.shape[0]))
print("Le tableau comporte {} colonne(s)".format(aide_alimentaire.shape[1]))
```

```
Le tableau comporte 1475 observation(s) ou article(s)
Le tableau comporte 4 colonne(s)
```

```
#Consulter le nombre de colonnes
print("Le tableau comporte {} colonnes".format(len(aide_alimentaire.columns)))
print("Les types de données dans chaque colonne sont
:\n{}".format(aide_alimentaire.dtypes))
print("Pour chaque colonne, voici le nombre de valeurs
:\n{}".format(aide_alimentaire.count()))
```

```
Le tableau comporte 4 colonnes
Les types de données dans chaque colonne sont :
Pays bénéficiaire    object
Année                int64
Produit              object
Valeur               int64
dtype: object
Pour chaque colonne, voici le nombre de valeurs :
Pays bénéficiaire    1475
Année                1475
Produit              1475
Valeur               1475
dtype: int64
```

```
#Affichage les 5 premières lignes de la table
aide_alimentaire.head()
```

```
border="1" class="dataframe">
```

	Pays bénéficiaire	Année	Produit	Valeur
0	Afghanistan	2013	Autres non- céréales	682
1	Afghanistan	2014	Autres non- céréales	335
2	Afghanistan	2013	Blé et Farin	39224
3	Afghanistan	2014	Blé et Farin	15160
4	Afghanistan	2013	Céréales	40504

```
#changement du nom de la colonne Pays bénéficiaire par Zone
aide_alimentaire=aide_alimentaire.rename(columns={'Pays bénéficiaire': 'Zone'})
```

```
#Multiplication de la colonne Aide_alimentaire qui contient des tonnes par 1000 pour
avoir des kg
aide_alimentaire['Valeur']=aide_alimentaire['Valeur']*1000
```

```
#Affichage les 5 premières lignes de la table
aide_alimentaire.head()
```

```
border="1" class="dataframe">
```

	Zone	Année	Produit	Valeur
0	Afghanistan	2013	Autres non-céréales	682000
1	Afghanistan	2014	Autres non-céréales	335000
2	Afghanistan	2013	Blé et Farin	39224000
3	Afghanistan	2014	Blé et Farin	15160000
4	Afghanistan	2013	Céréales	40504000

2.3 - Analyse exploratoire du fichier sous nutrition

```
#Afficher les dimensions du dataset
print("Le tableau comporte {} observation(s) ou
article(s)".format(sous_nutrition.shape[0]))
print("Le tableau comporte {} colonne(s)".format(sous_nutrition.shape[1]))
```

```
Le tableau comporte 1218 observation(s) ou article(s)
Le tableau comporte 3 colonne(s)
```

```
#Consulter le nombre de colonnes
print("Le tableau comporte {} colonne(s)".format(len(sous_nutrition.columns)))
print("Les types de données dans chaque colonne sont
:\n{}".format(sous_nutrition.dtypes))
print("Pour chaque colonne, voici le nombre de valeurs
:\n{}".format(sous_nutrition.count()))
```

```
Le tableau comporte 3 colonne(s)
Les types de données dans chaque colonne sont :
Zone      object
Année     object
Valeur    object
dtype: object
Pour chaque colonne, voici le nombre de valeurs :
Zone      1218
Année     1218
Valeur    624
dtype: int64
```

```
#Afficher les 5 premières lignes de la table
sous_nutrition.head()
```

```
border="1" class="dataframe">
```

	Zone	Année	Valeur
0	Afghanistan	2012-2014	8.6
1	Afghanistan	2013-2015	8.8
2	Afghanistan	2014-2016	8.9
3	Afghanistan	2015-2017	9.7
4	Afghanistan	2016-2018	10.5

```
#Conversion de la colonne sous nutrition en numérique
sous_nutrition['Valeur'] = sous_nutrition['Valeur'].replace('<0.1', None)
sous_nutrition['Valeur'] = sous_nutrition['Valeur'].astype(float)
```

```
#Conversion de la colonne (avec l'argument errors=coerce qui permet de convertir
automatiquement les lignes qui ne sont pas des nombres en NaN)
#Puis remplacement des NaN en 0
sous_nutrition['Valeur'] = pd.to_numeric(sous_nutrition['Valeur'], errors='coerce')
sous_nutrition = sous_nutrition.fillna(0)
sous_nutrition.head()
```

```
border="1" class="dataframe">
```

	Zone	Année	Valeur
0	Afghanistan	2012-2014	8.6
1	Afghanistan	2013-2015	8.8
2	Afghanistan	2014-2016	8.9
3	Afghanistan	2015-2017	9.7
4	Afghanistan	2016-2018	10.5

```
#changement du nom de la colonne Valeur par sous_nutrition
sous_nutrition=sous_nutrition.rename(columns={'Valeur': 'sous_nutrition'})
sous_nutrition.head()
```

```
border="1" class="dataframe">
```

	Zone	Année	sous_nutrition
0	Afghanistan	2012-2014	8.6
1	Afghanistan	2013-2015	8.8
2	Afghanistan	2014-2016	8.9
3	Afghanistan	2015-2017	9.7
4	Afghanistan	2016-2018	10.5

```
#Multiplication de la colonne sous_nutrition par 1000000
sous_nutrition['sous_nutrition']=sous_nutrition['sous_nutrition']*1000000
```

```
#Afficher les 5 premières lignes de la table
sous_nutrition['sous_nutrition'] = sous_nutrition['sous_nutrition'].astype(int)
sous_nutrition.head()
```

border="1" class="dataframe">

	Zone	Année	sous_nutrition
0	Afghanistan	2012-2014	8600000
1	Afghanistan	2013-2015	8800000
2	Afghanistan	2014-2016	8900000
3	Afghanistan	2015-2017	9700000
4	Afghanistan	2016-2018	10500000

3.1 - Proportion de personnes en sous nutrition

```
# Il faut tout d'abord faire une jointure entre la table population et la table sous nutrition, en ciblant l'année 2017
```

```
#Je filtre les deux tables concernées pour l'année 2017
sous_nutrition_2017 = sous_nutrition[sous_nutrition['Année'] == '2016-2018']
population_2017 = population[population['Année'] == 2017]
```

```
#Je fais la fusion
pop_sous_nutri = pd.merge(sous_nutrition_2017, population_2017, on='Zone',
how='inner')
```

```
#Affichage du dataset
pop_sous_nutri.head()
```

border="1" class="dataframe">

	Zone	Année_x	sous_nutrition	Année_y	Population
0	Afghanistan	2016-2018	10500000	2017	36296113
1	Afrique du Sud	2016-2018	3100000	2017	57009756
2	Albanie	2016-2018	100000	2017	2884169
3	Algérie	2016-2018	1300000	2017	41389189
4	Allemagne	2016-2018	0	2017	82658409

```
#Calcul et affichage du nombre de personnes en état de sous nutrition
```

```
#Je fais la somme des deux colonnes concernées
total_pop_sous_nutri_2017 = pop_sous_nutri['sous_nutrition'].sum()
total_pop_2017 = pop_sous_nutri['Population'].sum()
```

```
#Je fais le calcul pour trouver la part des personnes sous-nutries
part_pop_sous_nutri_mondial = round(total_pop_sous_nutri_2017/total_pop_2017*100, 1)
```

```
print(f"Il y a dans le monde {total_pop_sous_nutri_2017:,} personnes en état de sous-nutrition. Cela représente {part_pop_sous_nutri_mondial} % de la population mondiale.")
```

Il y a dans le monde 535,700,000 personnes en état de sous-nutrition. Cela représente 7.1 % de la population mondiale.

3.2 - Nombre théorique de personne qui pourrait être nourries

```
#Combien mange en moyenne un être humain ?
```

```
#Selon l'Oxfam qui relaie des chiffres des Etats-Unis, on peut retenir le chiffre de 2100 Kcal/jour par personne comme un seuil des besoins alimentaires essentiels.
```

```
#Source => https://www.oxfamfrance.org/agriculture-et-securite-alimentaire/faim-dans-le-monde-causes-consequences-solutions/#\_ftnref3
```

```
#On commence par faire une jointure entre le data frame population et Dispo_alimentaire afin d'ajouter dans ce dernier la population
```

```
population_2017 = population[population['Année'] == 2017]
```

```
dispo_alim_pop_merge = pd.merge(dispo_alimentaire, population_2017[['Zone', 'Population']], on='Zone', how='inner')
```

```
#Affichage du nouveau dataframe
```

```
dispo_alim_pop_merge.head()
```



```
#Création de la colonne dispo_kcal avec calcul des kcal disponibles mondialement
dispo_alim_pop_agrege = dispo_alim_pop_merge.pivot_table(
    index='Zone',
    aggfunc={
        'Disponibilité alimentaire (Kcal/personne/jour)': 'sum',
        'Population': 'first'
    }
)
dispo_alim_pop_agrege=dispo_alim_pop_agrege.rename(columns={'Disponibilité
alimentaire (Kcal/personne/jour)': 'Disponibilité alimentaire total
(Kcal/personne/jour)'})
dispo_alim_pop_agrege.head()
```

border="1" class="dataframe">

Zone	Disponibilité alimentaire total (Kcal/personne/jour)	Population
Afghanistan	2087.0	36296113
Afrique du Sud	3020.0	57009756
Albanie	3188.0	2884169
Algérie	3293.0	41389189
Allemagne	3503.0	82658409

```
#Calcul du nombre d'humains pouvant être nourris
```

```
#Rajout des deux colonnes 'Nombre de personnes pouvant être nourris' et 'Part de la
population'
```

```
dispo_alim_pop_agrege['Nombre de personnes pouvant être
nourris']=dispo_alim_pop_agrege['Disponibilité alimentaire total
(Kcal/personne/jour)']*dispo_alim_pop_agrege['Population'] / 2100
dispo_alim_pop_agrege['Nombre de personnes pouvant être
nourris']=dispo_alim_pop_agrege['Nombre de personnes pouvant être
nourris'].astype(int)
dispo_alim_pop_agrege['Part de la population']=round(dispo_alim_pop_agrege['Nombre
de personnes pouvant être nourris']/dispo_alim_pop_agrege['Population'] * 100,1)
dispo_alim_pop_agrege.head(5)
```

```
#Calcul du nombre de personnes à l'échelle mondiale
```

```
total_pop_mondiale=dispo_alim_pop_agrege['Population'].sum()
total_capacité_personnes_nourri = dispo_alim_pop_agrege['Nombre de personnes pouvant
être nourris'].sum()
part_mondial_pers_nutri =
round(total_capacité_personnes_nourri/total_pop_mondiale*100,1)
```

```
print(f"Au regard des capacités de production de nourriture, le nombre d'humains
pouvant être nourris est de {total_capacité_personnes_nourri}, soit
{part_mondial_pers_nutri} % de la population mondiale.")
```

Au regard des capacités de production de nourriture, le nombre d'humains pouvant être nourris est de 10086744073, soit 136.9 % de la population mondiale.

3.3 - Nombre théorique de personne qui pourrait être nourrie avec les produits végétaux

```
#Transfert des données avec les végétaux dans un nouveau dataframe
population_2017 = population[population['Année'] == 2017]
dispo_alimentaire_vegetaux=dispo_alimentaire[dispo_alimentaire['Origine'] ==
'vegetale']
population_2017['Population'].astype(int)

dispo_alimentaire_vegetaux = dispo_alimentaire_vegetaux.drop(columns=[
    "Aliments pour animaux",
    "Autres Utilisations",
    "Disponibilité alimentaire en quantité (kg/personne/an)",
    "Disponibilité de matière grasse en quantité (g/personne/jour)",
    "Disponibilité de protéines en quantité (g/personne/jour)",
    "Disponibilité intérieure", "Exportations - Quantité",
    "Importations - Quantité",
    "Nourriture",
    "Pertes",
    "Production",
    "Semences",
    "Traitement",
    "Variation de stock"])

pop_dispo_vegetaux_merge = pd.merge(dispo_alimentaire_vegetaux, population_2017, on
= 'Zone', how='left')

display(pop_dispo_vegetaux_merge)
```


border="1" class="dataframe">

	Zone	Produit	Origine	Disponibilité alimentaire (Kcal/personne/jour)	Année	Population
0	Afghanistan	Agrumes-Autres	vegetale	1.0	2017	36296113
1	Afghanistan	Aliments pour enfants	vegetale	1.0	2017	36296113
2	Afghanistan	Ananas	vegetale	0.0	2017	36296113
3	Afghanistan	Bananes	vegetale	4.0	2017	36296113
4	Afghanistan	Bière	vegetale	0.0	2017	36296113
...	...	...	...	...	...	...
11891	Îles Salomon	Sésame	vegetale	0.0	2017	636039
11892	Îles Salomon	Thé	vegetale	0.0	2017	636039
11893	Îles Salomon	Tomates	vegetale	0.0	2017	636039
11894	Îles Salomon	Vin	vegetale	0.0	2017	636039
11895	Îles Salomon	Épices-Autres	vegetale	4.0	2017	636039

11896 rows × 6 columns

```
#Calcul du nombre de kcal disponible pour les végétaux

pop_dispo_vegetaux_merge = pop_dispo_vegetaux_merge.pivot_table(
    index='Zone',
    aggfunc={
        'Disponibilité alimentaire (Kcal/personne/jour)': 'sum',
        'Population': 'first'
    })

display(pop_dispo_vegetaux_merge)
```

border="1" class="dataframe">

Zone	Disponibilité alimentaire (Kcal/personne/jour)	Population
Afghanistan	1871.0	36296113
Afrique du Sud	2533.0	57009756
Albanie	2203.0	2884169
Algérie	2915.0	41389189
Allemagne	2461.0	82658409
...	...	...
Émirats arabes unis	2718.0	9487203
Équateur	1732.0	16785361
États-Unis d'Amérique	2698.0	325084756
Éthiopie	2005.0	106399924
Îles Salomon	2187.0	636039

174 rows × 2 columns

```
#Calcul du nombre d'humains pouvant être nourris avec les végétaux

#Rajout des deux colonnes 'Nombre de personnes pouvant être nourris' et 'Part de la
population'
pop_dispo_vegetaux_merge['Nombre de personnes pouvant être
nourris']=pop_dispo_vegetaux_merge['Disponibilité alimentaire
(Kcal/personne/jour)']*pop_dispo_vegetaux_merge['Population'] / 2100
pop_dispo_vegetaux_merge['Nombre de personnes pouvant être
nourris']=pop_dispo_vegetaux_merge['Nombre de personnes pouvant être
nourris'].astype(int)
pop_dispo_vegetaux_merge['Part de la
population']=round(pop_dispo_vegetaux_merge['Nombre de personnes pouvant être
nourris']/pop_dispo_vegetaux_merge['Population'] * 100,1)

display(pop_dispo_vegetaux_merge.head(5))

#Calcul du nombre de personnes à l'échelle mondiale
total_pop_mondiale=dispo_alim_pop_agrege['Population'].sum()
total_capacité_personnes_nourri_vege = pop_dispo_vegetaux_merge['Nombre de personnes
pouvant être nourris'].sum()
part_mondial_pers_nutri_vege =
round(total_capacité_personnes_nourri_vege/total_pop_mondiale*100,1)

print(f"Au regard des capacités de production de nourriture, le nombre d'humains
pouvant être nourris est de {total_capacité_personnes_nourri_vege}, soit
{part_mondial_pers_nutri_vege} % de la population mondiale.")
```

Au regard des capacités de production de nourriture, le nombre d'humains pouvant être nourris est de 8309290114, soit 112.8 % de la population mondiale.

border="1" class="dataframe">

Zone	Disponibilité alimentaire (Kcal/personne/jour)	Population	Nombre de personnes pouvant être nourris	Part de la population
Afghanistan	1871.0	36296113	32338108	89.1
Afrique du Sud	2533.0	57009756	68764624	120.6
Albanie	2203.0	2884169	3025630	104.9
Algérie	2915.0	41389189	57452136	138.8
Allemagne	2461.0	82658409	96867783	117.2

3.4 - Utilisation de la disponibilité intérieure

```
#Calcul de la disponibilité totale
```

```
dispo_alim_mondiale = dispo_alimentaire.pivot_table(  
    index = 'Zone',  
    values = 'Disponibilité intérieure',  
    aggfunc = 'sum'  
)  
display(dispo_alim_mondiale)  
print(f"Le total de la disponibilité intérieure mondiale est de  
{dispo_alimentaire['Disponibilité intérieure'].sum():,} tonnes.")
```

```
Le total de la disponibilité intérieure mondiale est de 9,848,994,000,000 tonnes.
```

border="1" class="dataframe">

Zone	Disponibilité intérieure
Afghanistan	13515000000
Afrique du Sud	61256000000
Albanie	4758000000
Algérie	42630000000
Allemagne	162275000000
...	...
Émirats arabes unis	11850000000
Équateur	18614000000
États-Unis d'Amérique	777992000000
Éthiopie	44014000000
Îles Salomon	695000000

174 rows × 1 columns

```

# Liste des colonnes qu'on veut analyser
colonnes_a_ajouter = ['Nourriture', 'Pertes', 'Semences', 'Traitement', 'Aliments
pour animaux', 'Autres Utilisations' ]

# Étape 2 : pour chaque colonne dans notre liste (ex: 'Nourriture', 'Pertes')
for col in colonnes_a_ajouter:
    # On crée une table temporaire avec la somme de cette colonne par pays
    temp = dispo_alimentaire.pivot_table(
        index='Zone',
        values=col,
        aggfunc='sum'
    )

    # On crée une nouvelle colonne dans notre tableau principal avec le suffixe
    "_part"
    # Elle contient la part de cette catégorie (Nourriture, Pertes...) dans la dispo
    intérieure
    dispo_alim_mondiale[col + '_part'] = round(temp[col] /
dispo_alim_mondiale['Disponibilité intérieure']*100,1)

display(dispo_alim_mondiale)

```

border="1" class="dataframe">

Zone	Disponibilité intérieur e	Nourritur e_part	Pertes_p art	Semence s_part	Traiteme nt_part	Aliments pour animaux_ part	Autres Utilisatio ns_part
Afghanis tan	13515000 000	79.4	8.4	2.9	0.5	5.7	3.1
Afrique du Sud	61256000 000	48.7	3.6	0.4	37.3	8.7	1.4
Albanie	47580000 00	73.1	5.8	1.2	2.4	13.9	3.7
Algérie	42630000 000	74.4	8.8	1.1	1.5	10.2	4.0
Allemagne	16227500 0000	48.8	2.3	1.0	24.6	18.6	4.8
...	...	...	...	...	...	...	...
Émirats arabes unis	11850000 000	55.3	5.9	0.0	6.8	9.9	22.7
Équateur	18614000 000	46.1	3.8	0.7	33.3	6.4	10.3
États- Unis d'Amériq ue	77799200 0000	41.2	0.9	1.3	18.1	19.1	19.9
Éthiopie	44014000 000	75.8	5.1	1.5	7.8	1.6	8.3
Îles Salomon	69500000 0	52.1	0.9	0.0	12.9	0.0	34.1

174 rows × 7 columns

```
#Calcul de la moyenne au niveau mondial
```

```
display(dispo_alim_mondiale['Nourriture_part'].mean())
display(dispo_alim_mondiale['Aliments pour animaux_part'].mean())
display(dispo_alim_mondiale['Pertes_part'].mean())
display(dispo_alim_mondiale['Semences_part'].mean())
display(dispo_alim_mondiale['Traitement_part'].mean())
display(dispo_alim_mondiale['Autres Utilisations_part'].mean())
```

```
57.517241379310356
```

```
12.026436781609195
```

```
4.6545977011494255
```

```
1.4741379310344827
```

```
17.02931034482759
```

```
7.470689655172413
```

3.5 - Utilisation des céréales

```
#Création d'une liste avec toutes les variables
df_cereales = dispo_alimentaire.loc[(dispo_alimentaire["Produit"] == "Millet") |
(dispo_alimentaire["Produit"] == "Orge") | (dispo_alimentaire["Produit"] ==
"Seigle") |
                                (dispo_alimentaire["Produit"] == "Céréales, Autres") |
(dispo_alimentaire["Produit"] == "Blé") | (dispo_alimentaire["Produit"] == "Maïs") |
                                (dispo_alimentaire["Produit"] == "Avoine") |
(dispo_alimentaire["Produit"] == "Riz (Eq Blanchi)") | (dispo_alimentaire["Produit"]
== "Sorgho")]
```

```
#Création d'un dataframe avec les informations uniquement pour ces céréales
display(df_cereales)
```

[illegible]

	om	on																	
15 57 5	Îles Sal om on	Or ge	ve get ale	0.0	0.0 7	0.0 0	0.0 1	10 00 00 0	0	10 00 00 0	0	0	0	0	0	10 00 00 0	0	0	0
15 59 1	Îles Sal om on	Riz (Eq Bla nc hi)	ve get ale	62 3.0	63. 76	1.3 6	10. 90	49 00 00 00	0	47 00 00 00	36 00 00 00	10 00 00 0	30 00 00 0	0	0	0	12 00 00 00	0	
15 59 3	Îles Sal om on	Sor gh o	ve get ale	0.0	0.0 0	0.0 0	0.0 0	0	0	0	0	0	0	0	0	0	0	0	0

1325 rows × 18 columns

```
#Affichage de la proportion d'alimentation animale
df_cereales_animal_pivot = df_cereales.pivot_table(index='Zone', values=
['Disponibilité intérieure','Aliments pour animaux'], aggfunc='sum')

# Rajour d'une colonne indiquant la part par pays
df_cereales_animal_pivot['Part alimentation animale (%)'] = round(
(df_cereales_animal_pivot['Aliments pour animaux'] /
df_cereales_animal_pivot['Disponibilité intérieure']) * 100, 1)
display(df_cereales_animal_pivot)

# Calcul de la part mondiale
part_mondial_animal = df_cereales_animal_pivot['Aliments pour
animaux'].sum()/df_cereales_animal_pivot['Disponibilité intérieure'].sum()*100
print(f"La proportion des céréales consacrée à l'alimentation animale au niveau
mondial est de {round(part_mondial_animal,1)} %.")
```

La proportion des céréales consacrée à l'alimentation animale au niveau mondial est de 35.9 %.

border="1" class="dataframe">

Zone	Aliments pour animaux	Disponibilité intérieure	Part alimentation animale (%)
Afghanistan	560000000	730300000	7.7
Afrique du Sud	481600000	1548100000	31.1
Albanie	411000000	114500000	35.9
Algérie	433500000	1500900000	28.9
Allemagne	2424200000	3938000000	61.6
...	...	...	...
Émirats arabes unis	105200000	318200000	33.1
Équateur	84600000	340500000	24.8
États-Unis d'Amérique	14001900000	34273500000	40.9
Éthiopie	66700000	1700400000	3.9
Îles Salomon	0	6400000	0.0

174 rows × 3 columns

```
#Affichage de la proportion d'alimentation humaine (correction consigne, il y avait
deux fois alimentation animale)
df_cereales_humain_pivot = df_cereales.pivot_table(index='Zone', values=
['Disponibilité intérieure','Nourriture'], aggfunc='sum')

# Rajour d'une colonne indiquant la part par pays
df_cereales_humain_pivot['Part alimétation humaine (%)'] = round(
    (df_cereales_humain_pivot['Nourriture'] /
df_cereales_humain_pivot['Disponibilité intérieure']) * 100, 1)
display(df_cereales_humain_pivot)

# Calcul de la part mondial
part_mondial_humain =
df_cereales_humain_pivot['Nourriture'].sum()/df_cereales_humain_pivot['Disponibilité
intérieure'].sum()*100

print(f"La proportion des céréales consacrée à l'alimentation humaine au niveau
mondial est de {round(part_mondial_humain,1)} %.")
```

La proportion des céréales consacrée à l'alimentation humaine au niveau mondial est de 43.0 %.

border="1" class="dataframe">

Zone	Disponibilité intérieure	Nourriture	Part alimentation humaine (%)
Afghanistan	7303000000	5494000000	75.2
Afrique du Sud	15481000000	9499000000	61.4
Albanie	1145000000	474000000	41.4
Algérie	15009000000	8534000000	56.9
Allemagne	39380000000	9135000000	23.2
...	...	...	...
Émirats arabes unis	3182000000	1484000000	46.6
Équateur	3405000000	1504000000	44.2
États-Unis d'Amérique	342735000000	33610000000	9.8
Éthiopie	17004000000	11663000000	68.6
Îles Salomon	64000000	50000000	78.1

174 rows × 3 columns

3.6 - Pays avec la proportion de personnes sous-alimentée la plus forte en 2017

```
#Création de la colonne proportion par pays
pop_sous_nutri['Part_des_personnes_sous_nutries'] =
(pop_sous_nutri['sous_nutrition']/pop_sous_nutri['Population']*100).round(1)
pop_sous_nutri = pop_sous_nutri.drop(columns=['Année_x', 'Année_y'])
pop_sous_nutri.head()
```

border="1" class="dataframe">

	Zone	sous_nutrition	Population	Part_des personnes_sous _nutries
0	Afghanistan	10500000	36296113	28.9
1	Afrique du Sud	3100000	57009756	5.4
2	Albanie	100000	2884169	3.5
3	Algérie	1300000	41389189	3.1
4	Allemagne	0	82658409	0.0

```
#affichage après trie des 10 pires pays
pop_sous_nutri.sort_values(by='Part_des_personnes_sous_nutries',
ascending=False).head(10)
```

border="1" class="dataframe">

	Zone	sous_nutrition	Population	Part_des personnes_sous _nutries
78	Haïti	5300000	10982366	48.3
157	République populaire démocratique de Corée	12000000	25429825	47.2
108	Madagascar	10500000	25570512	41.1
103	Libéria	1800000	4702226	38.3
100	Lesotho	800000	2091534	38.2
183	Tchad	5700000	15016753	38.0
161	Rwanda	4200000	11980961	35.1
121	Mozambique	9400000	28649018	32.8
186	Timor-Leste	400000	1243258	32.2
0	Afghanistan	10500000	36296113	28.9

3.7 - Pays qui ont le plus bénéficié d'aide alimentaire depuis 2013

```
#calcul du total de l'aide alimentaire par pays
aide_alim_total = aide_alimentaire.pivot_table(index = 'Zone', values = 'Valeur',
aggfunc='sum')
display(aide_alim_total)
```

border="1" class="dataframe">

Zone	Valeur
Afghanistan	185452000
Algérie	81114000
Angola	5014000
Bangladesh	348188000
Bhoutan	2666000
...	...
Zambie	3026000
Zimbabwe	62570000
Égypte	1122000
Équateur	1362000
Éthiopie	1381294000

76 rows × 1 columns

```
#affichage après trie des 10 pays qui ont bénéficié le plus de l'aide alimentaire
aide_alim_total['Valeur'] = aide_alim_total['Valeur'] / 1_000_000
aide_alim_total.sort_values(by = 'Valeur', ascending=False).head(10)
```

```
border="1" class="dataframe">
```

Zone	Valeur
République arabe syrienne	1858.943
Éthiopie	1381.294
Yémen	1206.484
Soudan du Sud	695.248
Soudan	669.784
Kenya	552.836
Bangladesh	348.188
Somalie	292.678
République démocratique du Congo	288.502
Niger	276.344

```
print(f"L'aide alimentaire mondiale versée depuis 2016 s'élève à
{round(aide_alim_total['Valeur'].sum(),1)} millions de tonnes")
```

L'aide alimentaire mondiale versée depuis 2016 s'élève à 11035.9 millions de tonnes

3.8 - Evolution des 5 pays qui ont le plus bénéficiés de l'aide alimentaire entre 2013 et 2016

```
#Création d'un dataframe avec la zone, l'année et l'aide alimentaire puis groupby
sur zone et année
```

```
aide_alim_zone_année = aide_alimentaire.groupby(['Zone', 'Année']).sum()
aide_alim_zone_année = aide_alim_zone_année.drop(columns=['Produit'])
aide_alim_zone_année.head(10)
```

border="1" class="dataframe">

		Valeur
Zone	Année	
Afghanistan	2013	128238000
	2014	57214000
Algérie	2013	35234000
	2014	18980000
	2015	17424000
Angola	2016	9476000
	2013	5000000
Bangladesh	2014	14000
	2013	131018000
	2014	194628000

```
#Création d'une liste contenant les 5 pays qui ont le plus bénéficiées de l'aide alimentaire
top_5_aide_alim = aide_alim_zone_année.groupby(['Zone']).sum().sort_values(by = 'Valeur', ascending=False).head(5)
top_5_pays = ['République arabe syrienne', 'Éthiopie', 'Yémen', 'Soudan du Sud', 'Soudan']
display(top_5_aide_alim)
```

border="1" class="dataframe">

	Valeur
Zone	
République arabe syrienne	1858943000
Éthiopie	1381294000
Yémen	1206484000
Soudan du Sud	695248000
Soudan	669784000

```
#On filtre sur le dataframe avec notre liste
aide_alim_zone_année = aide_alim_zone_année.reset_index()
aide_top_5_filtre =
aide_alim_zone_année[aide_alim_zone_année['Zone'].isin(top_5_pays)]
display(aide_top_5_filtre)
```

border="1" class="dataframe">

	Zone	Année	Valeur
159	République arabe syrienne	2013	563566000
160	République arabe syrienne	2014	651870000
161	République arabe syrienne	2015	524949000
162	République arabe syrienne	2016	118558000
191	Soudan	2013	330230000
192	Soudan	2014	321904000
193	Soudan	2015	17650000
194	Soudan du Sud	2013	196330000
195	Soudan du Sud	2014	450610000
196	Soudan du Sud	2015	48308000
214	Yémen	2013	264764000
215	Yémen	2014	103840000
216	Yémen	2015	372306000
217	Yémen	2016	465574000
225	Éthiopie	2013	591404000
226	Éthiopie	2014	586624000
227	Éthiopie	2015	203266000

```
# Affichage des pays avec l'aide alimentaire par année
aide_top_5_filtre_pv = aide_top_5_filtre.pivot_table(index = 'Zone', columns =
'Année', values = 'Valeur', aggfunc='sum')
aide_top_5_filtre_pv = aide_top_5_filtre_pv.fillna(0).astype(int)
display(aide_top_5_filtre_pv)
```

border="1" class="dataframe">

Année	2013	2014	2015	2016
Zone				
République arabe syrienne	563566000	651870000	524949000	118558000
Soudan	330230000	321904000	17650000	0
Soudan du Sud	196330000	450610000	48308000	0
Yémen	264764000	103840000	372306000	465574000
Éthiopie	591404000	586624000	203266000	0

3.9 - Pays avec le moins de disponibilité par habitant

```
#Calcul de la disponibilité en kcal par personne par jour par pays
dispo_alim_pop_merge_epure = dispo_alim_pop_merge.drop(columns = ['Produit',
'Origine', 'Disponibilité alimentaire en quantité (kg/personne/an)',
'Disponibilité de matière grasse en quantité
(g/personne/jour)', 'Disponibilité de protéines en quantité (g/personne/jour)',
'Disponibilité intérieure', 'Exportations -
Quantité', 'Importations - Quantité', 'Nourriture', 'Pertes', 'Production',
'Semences', 'Traitement', 'Aliments pour
animaux', 'Autres Utilisations', 'Variation de stock', 'Population'
])
dispo_alim_par_pays = dispo_alim_pop_merge_epure.groupby('Zone').sum()
display(dispo_alim_par_pays)
```

border="1" class="dataframe">

Zone	Disponibilité alimentaire (Kcal/personne/jour)
Afghanistan	2087.0
Afrique du Sud	3020.0
Albanie	3188.0
Algérie	3293.0
Allemagne	3503.0
...	...
Émirats arabes unis	3275.0
Équateur	2346.0
États-Unis d'Amérique	3682.0
Éthiopie	2129.0
Îles Salomon	2383.0

174 rows × 1 columns

```
#Affichage des 10 pays qui ont le moins de dispo alimentaire par personne
dispo_alim_par_pays.sort_values('Disponibilité alimentaire
(Kcal/personne/jour)').head(10)
```

border="1" class="dataframe">

Zone	Disponibilité alimentaire (Kcal/personne/jour)
République centrafricaine	1879.0
Zambie	1924.0
Madagascar	2056.0
Afghanistan	2087.0
Haïti	2089.0
République populaire démocratique de Corée	2093.0
Tchad	2109.0
Zimbabwe	2113.0
Ouganda	2126.0
Timor-Leste	2129.0

3.10 - Pays avec le plus de disponibilité par habitant

```
#Affichage des 10 pays qui ont le plus de dispo alimentaire par personne  
dispo_alim_par_pays.sort_values(by='Disponibilité alimentaire (Kcal/personne/jour)',  
ascending=False).head(10)
```

border="1" class="dataframe">

Zone	Disponibilité alimentaire (Kcal/personne/jour)
Autriche	3770.0
Belgique	3737.0
Turquie	3708.0
États-Unis d'Amérique	3682.0
Israël	3610.0
Irlande	3602.0
Italie	3578.0
Luxembourg	3540.0
Égypte	3518.0
Allemagne	3503.0

3.11 - Exemple de la Thaïlande pour le Manioc


```
#création d'un dataframe avec uniquement la Thaïlande
dispo_alim_thai_manioc = dispo_alimentaire[(dispo_alimentaire['Zone'] ==
'Thaïlande')&
                                            (dispo_alimentaire['Produit'] == 'Manioc')
                                            ]
display(dispo_alim_thai_manioc)
```

border="1" class="dataframe">

Zone	Produit	Origine	Disponibilité alimentaire (Kcal/personne/jour)	Disponibilité alimentaire en quantité (kg/personne/jour)	Disponibilité de matière grasse en quantité (g/personne/jour)	Disponibilité de protéines en quantité (g/personne/jour)	Disponibilité énergétique	Disponibilité	Exportations	Importations	Nourriture	Pertes	Production	Semences	Traitement	Aliments pour animaux	Autres Utilisations	Variation de stock
								intérieure	- Quantité	- Quantité								
13809	Thaïlandais	Manioc	végétale	40.0	13.0	0.05	0.14	62640000	252140000	12500000	8710000	15110000	302280000	0	0	18000000	20810000	0

```
#Calcul de la sous nutrition en Thaïlande
```

```
sous_nutri_thai_2017 = sous_nutrition[(sous_nutrition['Zone'] == 'Thaïlande')&
(sous_nutrition['Année'] == '2016-2018')
].iloc[0]['sous_nutrition']

pop_thaïlande_2017 = population[(population['Zone'] == 'Thaïlande')&
(population['Année'] == 2017)
].iloc[0]['Population']

print(f"La population en état de sous-nutrition en Thaïlande est de
{sous_nutri_thai_2017} personnes, soit
{round((sous_nutri_thai_2017)/(pop_thaïlande_2017)*100,1)} % de sa population.")
```

La population en état de sous-nutrition en Thaïlande est de 6200000 personnes, soit 9.0 % de sa population.

```
# On calcule la proportion exportée en fonction de la proportion (production ?)
part_exportation = dispo_alim_thai_manioc['Exportations -
Quantité']/dispo_alim_thai_manioc['Production']*100
export_thai = round(part_exportation.iloc[0], 1)
print(f"La part de la production qui est exportée est de {export_thai} %.")
```

La part de la production qui est exportée est de 83.4 %.

```
dispo_alim_thai = dispo_alimentaire[(dispo_alimentaire['Zone'] == 'Thaïlande')]
dispo_intérieure_total_thai = dispo_alim_thai['Disponibilité intérieure'].sum()
dispo_intérieure_kcal_total_thai = dispo_alim_thai['Disponibilité alimentaire
(Kcal/personne/jour)'].sum()
dispo_intérieure_manioc_thai =
dispo_alim_thai_manioc[dispo_alim_thai_manioc['Produit'] == 'Manioc'].iloc[0]
['Disponibilité intérieure']

part_manioc_dispo_int_total_thai = dispo_intérieure_manioc_thai /
dispo_intérieure_total_thai*100
print(f"La part du manioc disponible dans la disponibilité totale intérieure du pays
est de {round(part_manioc_dispo_int_total_thai,1)} %.")
```

La part du manioc disponible dans la disponibilité totale intérieure du pays est de 4.0 %.

Etape 6 - Analyse complémentaires

Analyse aide alimentaire par habitant

```
#J'importe un dataframe précédemment créé
aide_alim_zone_année = aide_alimentaire.groupby(['Zone', 'Année']).sum()
aide_alim_zone_année = aide_alim_zone_année.drop(columns=['Produit'])

#Je crée un dataframe pour le fichier population avec le même format
population_pays_année = population.groupby(['Zone', 'Année']).sum()

# Je fusionne les deux
population_aide_alimentaire_pays_année = pd.merge(aide_alim_zone_année,
population_pays_année, on=['Zone', 'Année'], how='inner')

#Je rajoute une colonne avec la calcul du ration en kilo d'aide alimentaire par
habitant
population_aide_alimentaire_pays_année['Aide par habitant en
kg']=round(population_aide_alimentaire_pays_année['Valeur']/population_aide_aliment
aire_pays_année['Population'],5)

#Suppression d'une valeur égale à 0
population_aide_alimentaire_pays_année =
population_aide_alimentaire_pays_année[population_aide_alimentaire_pays_année['Vale
ur'] != 0]

population_aide_alimentaire_pays_année.head(5)
```

border="1" class="dataframe">

		Valeur	Population	Aide par habitant en kg
Zone	Année			
Afghanistan	2013	128238000	32269589	3.97396
	2014	57214000	33370794	1.71449
Algérie	2013	35234000	38140133	0.92380
	2014	18980000	38923692	0.48762
	2015	17424000	39728025	0.43858

```
population_aide_alimentaire_pays_année.sort_values(by = 'Aide par habitant en  
kg').head(10)
```

border="1" class="dataframe">

		Valeur	Population	Aide par habitant en kg
Zone	Année			
Angola	2014	14000	26941779	0.00052
Bolivie (État plurinational de)	2014	6000	10706517	0.00056
Chine, continentale	2014	1298000	1399453965	0.00093
Cuba	2014	14000	11306904	0.00124
Burkina Faso	2016	72000	18646357	0.00386
Iran (République islamique d')	2015	492000	78492215	0.00627
	2016	582000	79563989	0.00731
Géorgie	2013	34000	4048701	0.00840
	2014	36000	4035311	0.00892
Égypte	2013	1122000	88404646	0.01269

```
population_aide_alimentaire_pays_année.sort_values(by = 'Aide par habitant en  
kg',ascending = False).head(10)
```

border="1" class="dataframe">

		Valeur	Population	Aide par habitant en kg
Zone	Année			
Soudan du Sud	2014	450610000	10554880	42.69210
République arabe syrienne	2014	651870000	18710711	34.83940
	2015	524949000	17997408	29.16803
	2013	563566000	19578461	28.78500
Djibouti	2013	20368000	883293	23.05917
	2014	17088000	898695	19.01424
Soudan du Sud	2013	196330000	10355035	18.95986
Palestine	2014	83134000	4428639	18.77191
Yémen	2016	465574000	27168208	17.13672
	2015	372306000	26497889	14.05040

--